

HomeGuard: A Real-Time Traffic Surveillance System for Enhanced Security

MS-DS - DTSA 5506

Pui Ho Cheung
College of Engineering and Applied Science
University of Colorado Boulder
Boulder Colorado, United States
Pui.cheung@colorado.edu

ABSTRACT

The HomeGuard project aims to develop a real-time traffic surveillance system that utilizes lightweight image recognition models to identify vehicles and potential threats in residential areas. By integrating a surveillance camera with advanced image processing capabilities, the system will provide homeowners with timely alerts regarding suspicious activities, enhancing security and peace of mind.

CCS CONCEPTS

- Computer Vision
- Machine Learning
- Security and Privacy
- Embedded Systems
- Human-Computer Interactions

KEYWORDS

Home Surveillance, Deep Learning, Image Recognition

ACM Reference format:

Pui Ho Cheung. 2025. HomeGuard: A Real-Time Traffic Surveillance System for Enhanced Security. In *Proceedings of ACM Woodstock conference (WOODSTOCK'18)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/1234567890>

1 Introduction

In recent years, the demand for home surveillance systems has surged, driven by increasing concerns over safety and security in residential areas. Traditional surveillance solutions often rely on cloud-based services, which can raise issues related to privacy, data control, and ongoing subscription costs. As a result, many homeowners are turning to self-hosted models that offer greater autonomy and flexibility in managing their surveillance needs. As the landscape of home surveillance continues to evolve, the shift towards lightweight self-hosted models represents a significant trend that empowers homeowners to take control of their security. By embracing these technologies, individuals can enhance their peace of mind while ensuring that their privacy and data remain protected.

2 Problem Statement

The HomeGuard project addresses a critical need for enhanced security in residential areas, where traditional surveillance systems often fall short in providing real-time analysis and alerts.

1. **Identification of Threats:** Homeowners face challenges in identifying potential threats from vehicles in their vicinity, leading to delayed responses to suspicious activities. Current

systems may not effectively analyze vehicle types or behaviors, resulting in missed opportunities for timely intervention.

2. **Inefficiency of Traditional Systems:** Many existing surveillance systems lack the capability to process and analyze video feeds in real-time, which can lead to a reliance on manual monitoring and delayed alerts. This inefficiency can compromise the safety of residents.
3. **Need for Lightweight Solutions:** With the increasing demand for smart home technologies, there is a need for lightweight image recognition models that can operate on limited hardware, such as Raspberry Pi, without sacrificing accuracy or speed. This is particularly important for homeowners who may not have access to high-end computing resources. Besides, users can utilize old smartphones as cameras and save costs of buying expensive surveillance cameras and still achieve the same functionality.
4. **Consequences of Inaction:** Failure to address these issues can result in increased vulnerability to crime, reduced peace of mind for homeowners, and a lack of trust in existing security solutions. The negative implications of inadequate surveillance can extend beyond individual households, affecting community safety.

By addressing these problems, the HomeGuard project aims to provide a simple solution that enhances home security through real-time vehicle surveillance and threat identification.

3 Project Timeline

The project is organized into a five-week timeline, divided into four main phases: Research, Data Preparation, Model Training, and Final Proposal.

In Week 1, activities included gathering vehicle images and relevant literature to build a comprehensive dataset. This foundational work is crucial for the subsequent stages of the project.

Week 2 is dedicated to Data Preparation. This phase involved conducting Exploratory Data Analysis, followed by data cleaning and preprocessing. These tasks are essential to ensure the quality and accuracy of the dataset for further studies.

The Model Training phase spanned Weeks 3 and 4. During this period, I developed the image recognition model and conducted initial testing. Additionally, multiple models were evaluated to determine the most effective approach.

Finally, Week 5 was allocated to the Final Proposal phase. In this concluding stage, I drafted the final proposal and prepared the final presentation.

4 Literature Review

Prior research has demonstrated the effectiveness of traffic monitoring systems in urban environments, utilizing various types of cameras and sensors to improve traffic flow and safety. Studies indicate that traffic surveillance cameras can reduce accidents by providing real-time data to authorities, which can also be adapted for residential use¹.

There are existing open-source tools for home and self-hosted surveillance systems, for example, Frigate², one of the most popular all-rounded monitoring frameworks for home cameras. While their functionality is excellent for general usage monitoring, for specific target for identifying vehicles, the resources required would be exceedingly how smaller and old devices are capable of.

Additionally, advancements in lightweight deep learning models have made it feasible to deploy image recognition systems on devices with limited processing power, making them suitable for home surveillance applications.

5 System Architecture

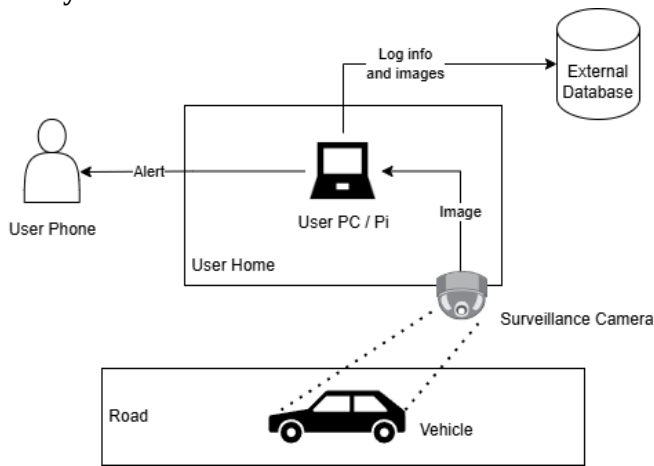


Figure 1: Proposed system architecture for HomeGuard

The proposed system architecture for monitoring the road outside a home using a surveillance camera involves several key components and processes (**Figure 1**). This architecture is designed to detect the presence of vehicles, identify their types, and log relevant data for user notifications.

1. Surveillance Camera Setup

A high-definition surveillance camera is installed to continuously monitor the road outside the home. The camera captures real-time video feeds, which are essential for detecting vehicles.

2. Vehicle Detection and Classification

The system employs a machine learning model, such as MobileNetV2, to analyze the video feed from the surveillance camera. This model is trained to detect the presence of vehicles and classify them into different types. The detection process involves processing each frame of the video feed to identify vehicles based on predefined criteria.

3. Image Input to User PC or Raspberry Pi

The captured images from the surveillance camera are sent to a user PC or a Raspberry Pi for further processing. This setup allows for local processing of the video feed, reducing latency and improving response times.

4. Perform Prediction

The machine learning model performs predictions on the incoming images to determine whether a vehicle is present and, if so, its type. This prediction is based on the features extracted from the images.

5. Logging Data into Database

For each detected vehicle, the system logs the following information into a database:

- The image of the vehicle
- A timestamp indicating when the vehicle was detected
- The result of the prediction (i.e., the type of vehicle)

This logging process ensures that there is a historical record of vehicle presence, which can be useful for analysis and reporting.

6. Threshold Monitoring and User Alerts

- The system continuously monitors the presence of vehicles. If a vehicle is detected and its presence exceeds a user-defined threshold (e.g., a specific duration of time), the system triggers an alert.
- Alerts can be sent to the user via various methods, such as push notifications, emails, or SMS, informing them of the prolonged presence of a vehicle outside their home.

6 Methodology

6.1 Data Selection

For the HomeGuard project, the dataset chosen for training the image recognition model is the "Vehicle Type Recognition" dataset available on Kaggle³. This dataset contains a collection of 400 vehicle images categorized into various types, including cars, buses, trucks, and motorcycles. The dataset is well-suited for our project as it provides a comprehensive representation of different vehicle classes that are commonly encountered in residential areas (**Figure 2**).

The dataset will be split into training, validation, and test sets (A 8:1:1 split). The training set will be used to train the model, the validation set will help tune hyperparameters and prevent overfitting, and the test set will be used to evaluate the model's performance on unseen data. Data augmentation techniques, such as rotation, flipping, and scaling, will be applied to enhance the diversity of the training data and improve the model's robustness.



Figure 2: Sample of Motorcycle images from the dataset

¹ Example of commercial level surveillance system from Avigilon: <https://www.avigilon.com/blog/traffic-control-cameras>

² Frigate is a complete and local NVR designed for Home Assistant with AI object detection. Uses OpenCV and Tensorflow to perform real-time object detection locally for IP cameras. <https://github.com/blakeblackshear/frigate>

³ Source: <https://www.kaggle.com/datasets/kaggleashwin/vehicle-type-recognition/data>

6 Methodology (continued)

6.2 Model Selection

In the HomeGuard project, we conducted a comparative analysis of several model options for vehicle recognition, including both a homebrew CNN and various pre-trained models. The goal was to identify the most effective and efficient model for our specific application in home surveillance.

6.2.1 Homebrew CNN

- Customization: The homebrew CNN was designed specifically for our dataset, allowing for tailored architecture adjustments based on the characteristics of the vehicle images.
- Simplicity: This model was straightforward to implement and required fewer resources initially, making it a good starting point for understanding the dataset and as a baseline for evaluating.

6.2.2 Pre-trained Models

- Robustness: Pre-trained models are known for their robustness and have been trained on large datasets, which allows them to recognize a wide range of features effectively.
- Transfer Learning: Utilizing pre-trained models allows us to leverage the knowledge gained from extensive training on diverse datasets, potentially improving performance on our specific task.
- Established Performance: These models have demonstrated high accuracy in various image classification tasks, making them reliable choices for initial testing.

The models I have selected include:

1. **ResNet50**: known as the **Residual Neural Network**, a widely used convolutional neural network known for its deep architecture, which allows it to learn complex features effectively. It has been benchmarked to provide excellent performance across various applications, making it a strong candidate for vehicle recognition tasks. ResNet50 is particularly effective in scenarios where high accuracy is required, as it has shown to outperform other models in many lab applications⁴.
2. **EfficientNet**: designed to optimize both accuracy and efficiency by scaling up the model size while maintaining a balance between depth, width, and resolution⁴.
3. **MobileNetV2**: a lightweight model that is particularly well-suited for mobile and edge devices. It is designed for speed and efficiency, making it faster to run and train compared to more complex models like ResNet50. MobileNetV2 has been shown to perform well even with limited data, as its simpler architecture helps prevent overfitting, allowing it to generalize better in certain scenarios⁵.

6.3 Model Training and Evaluation

The training process involves several key steps:

1. **Data Preprocessing**: The images will be resized to a uniform dimension (e.g., 224x224 pixels). Normalization will be applied to scale pixel values to a range of [0, 1], which helps improve model convergence during training.
2. **Training the Model**: The model will be trained using the training dataset, employing techniques such as early stopping and learning rate scheduling to optimize performance. The loss function used will be categorical cross-entropy, suitable for multi-class classification tasks.
3. **Validation**: During training, the model's performance will be evaluated on the validation set to monitor accuracy and loss. This will help in tuning hyperparameters and preventing overfitting.

4. **Testing**: After training, the model will be evaluated on the test set to assess its generalization capabilities. Key evaluation metrics will include accuracy, precision, recall, F1 score, and confusion matrix.

6.4 Real-time simulation for Extra Evaluation

The model selected for the surveillance system must not only be accurate, light weight but also fast in terms of speed of prediction. Therefore, to simulate the actual environment of implementing the model prediction engine on a tiny device (e.g. Raspberry Pi, old personal computers), the model will be deployed on a controlled Docker image, where it will process images. Here is the workflow for testing the model performance (**Figure 3**):

1. **API Server Endpoint for Images**: The process begins with an API server that stores vehicle images. This endpoint is responsible for serving vehicle images that will be processed and analyzed by the HomeGuard application.
2. **Scheduled Request**: The HomeGuard application periodically sends a scheduled request to the API server endpoint to retrieve new images.
3. **CNN Model**: Once the images are retrieved, they are processed by a CNN model housed within a Docker container. The CNN model is responsible for analyzing the images and making predictions.
4. **Log to Database**: After the CNN model processes the images and generates predictions, the HomeGuard application logs the information and images to an external database. This step ensures that all data and predictions are stored for future reference and analysis.
5. **Alert Send to Email**: Finally, if certain conditions are met based on the predictions (e.g., detection of a specific type of vehicle), the HomeGuard application sends an email alert to a simulated email mailbox.

The system will be evaluated based on its ability to accurately detect and classify vehicles in real-time, as well as the response time for generating alerts when suspicious vehicles are identified. (**Figure 4**)

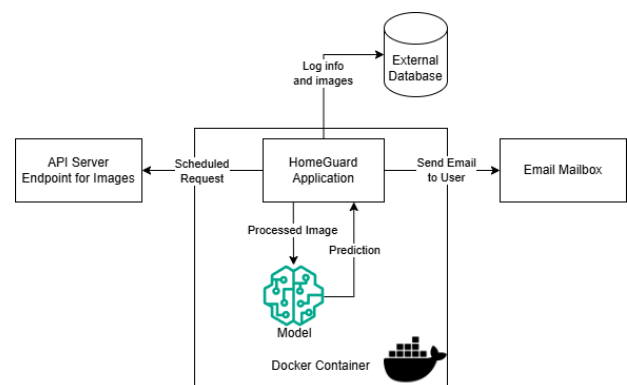


Figure 3: Workflow Design for Simulation

⁴ Source: [https://github.com/DeepLabCut/DeepLabCut/wiki/What-neural-network-should-I-use%3F-\(Trade-offs,-speed-performance,-and-considerations\)](https://github.com/DeepLabCut/DeepLabCut/wiki/What-neural-network-should-I-use%3F-(Trade-offs,-speed-performance,-and-considerations))

⁵ Source: <https://medium.com/@imtiazhmed2206/why-your-mobilenetv2-model-performs-better-than-resnet50-2a9998fda4c7>

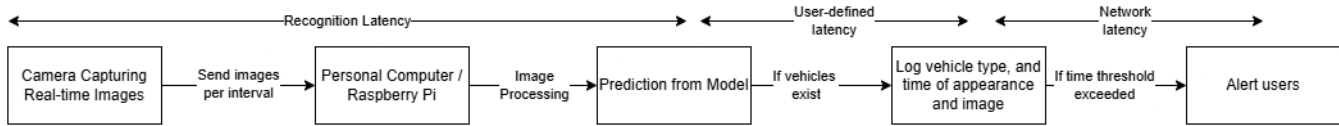


Figure 4: Importance of latency in HomeGuard Design: the goal is to reduce recognition latency within detection and prediction.

7 Results

7.1 Model Performance

Model	Accuracy (Best)		Loss (Best)	
	Train	Validation	Train	Validation
CNN	0.59	0.60	1.05	0.80
ResNet50	0.45	0.60	1.24	1.10
EfficientNet	0.97	0.23	2.21	4.40
MobileNetV2	0.96	0.98	0.11	0.12

Figure 5: Model Accuracies and Losses from CNN models used

After training the models mentioned before we collect the evaluation metrics (**Figure 5**). Here are the results:

1. **Homebrew CNN:** This model was designed with a few convolutional layers and pooling layers, aiming to achieve a balance between complexity and performance. However, it struggled to generalize well on unseen data as either accuracy or loss are performing well. This was expected as it is served as the baseline model.

Then we explored several pre-trained models, these models are known for their robustness and have been trained on large datasets, which allow them to recognize a wide range of features.

2. **ResNet50:** The ResNet50 model demonstrates the weakest performance among the models, with training accuracy of 0.45 and validation accuracy of 0.60. The high loss values suggest significant learning challenges. This could be because ResNet50's deep architecture might be too complex for the small dataset, leading to poor feature extraction. The model appears to be underfitting, indicating it's struggling to capture the essential characteristics of vehicle classes.

3. **EfficientNet:** The EfficientNet model shows an interesting pattern with extremely high training accuracy (0.97) but low validation accuracy (0.23). The high training accuracy coupled with dramatically lower validation accuracy is a clear sign of severe overfitting. The high losses further confirm this. This suggests the model has memorized the training data rather than learning generalizable features, which would make it perform poorly on new, unseen images.

4. **MobileNetV2:** The MobileNetV2 model demonstrates the most promising results, with high and consistent accuracy across training (0.96) and validation (0.98) sets. The low loss indicate excellent learning and generalization. This suggests that MobileNetV2's efficient architecture is well-suited to the small dataset, successfully capturing meaningful features while avoiding overfitting. It appears to be the most reliable model for vehicle classification in this scenario.

After evaluating the performance of the homebrew CNN and the pre-trained models, we ultimately selected MobileNetV2 for our project. MobileNetV2 is a lightweight model specifically designed for mobile and edge devices, making it ideal for real-time applications like home surveillance. Its architecture, which employs depth wise separable convolutions, allows for efficient processing while maintaining high accuracy. This makes MobileNetV2 a suitable choice for our needs, balancing performance and resource constraints effectively.

7.2 Hyperparameter Tuning

Classification Report:				
	precision	recall	f1-score	support
Bus	0.60	0.86	0.71	7
Car	1.00	1.00	1.00	7
Truck	0.86	0.60	0.71	10
motorcycle	1.00	1.00	1.00	8
accuracy			0.84	32
macro avg	0.86	0.86	0.85	32
weighted avg	0.87	0.84	0.84	32

Classification Report:				
	precision	recall	f1-score	support
Bus	1.00	0.86	0.92	7
Car	1.00	1.00	1.00	7
Truck	0.91	1.00	0.95	10
motorcycle	1.00	1.00	1.00	8
accuracy			0.97	32
macro avg	0.98	0.96	0.97	32
weighted avg	0.97	0.97	0.97	32

Figure 6: Classification Report of the First Model (Up) versus Best Model (Down) from Hyperparameter Tuning

For this vehicle classification project with a small dataset of 100 images per class, hyperparameter tuning would be crucial to optimize model performance. Potential strategies could include experimenting with dropout rates to prevent overfitting, especially given the limited training data. Learning rate exploration might involve starting with lower rates and using adaptive optimizers like Adam or RMSprop to improve convergence.

For the network architecture, gradually increasing dense layers (1-3 layers) with varying neuron counts could help capture more complex features. Pooling strategies like max pooling with 2x2 or 3x3 kernel sizes could be tested to effectively reduce spatial dimensions while retaining important feature information.

The result from hyperparameter tuning is promising (**Figure 6**). The model performs best with a low learning rate but with simple layer architecture, with much higher F1-score across 4 classes in general. The possible reason behind is small dataset's inherent limitations, where complex architectures are prone to overfitting. With only 100 images per class, a simpler model with fewer parameters can more

effectively generalize and capture the essential distinguishing features of vehicle classes. The low learning rate allows for more precise and gradual weight updates, preventing the model from quickly adapting to noise in the limited training data.

7.3 Latency Test from Simulation

Environment	Average	Best	Standard Deviation
Jupyter Notebook	0.85	0.66	0.13
Docker Container	1.22	1.03	0.49

Figure 7: Latency Test Results (Unit in seconds)

We deployed our model using the simulation setup. When deployed in a Jupyter Notebook environment, the model demonstrates an average prediction latency of 0.85 seconds, with the best performance reaching 0.66 seconds and a relatively low standard deviation of 0.13. In contrast, the Docker container environment shows higher latency, with an average of 1.22 seconds, a best-case scenario of 1.03 seconds, and a significantly larger standard deviation of 0.49. The higher standard deviation in the Docker setup suggests less consistent prediction times, which could impact real-time applications requiring predictable inference speeds. However, in real life scenarios, when users want the best performance from our surveillance system, it highly depends on the user-defined latency threshold on how frequently the model performs predictions on the live-feed images. If the frequency is set too high, the model may not react fast enough in a low resource environment, like Raspberry Pi or container, for example. This result shows that the latency threshold should not be set too low, but even with 3 seconds of latency, it is still acceptable in actual implementation.

8 Limitations

From the model evaluations, the chosen model performance is within our expectation. However, this project has a couple of limitations from multiple dimensions:

- Limited dataset size:** our dataset only contains 100 images of each type of vehicle, which may not have a large enough variety of real-life vehicles.
- Restricted computation resources:** this project was performed on a personal computer without many GPU resources to train our models, which are essential to tasks like image recognition. Therefore, most CNN models are restricted to low Epoch count and low number of layers, which may result in worse model performance.
- Training assumption:** our dataset only contains images of four types of vehicles. This restricted variety may not accurately represent real-life scenarios, where vehicles may not always be present outside homes. Consequently, the training dataset primarily consists of images with vehicles, which could lead to confusion in the prediction model when it encounters images without vehicles

9 Future Work

Given the limitations identified in our project, several possible cases for future work can be pursued to enhance the vehicle recognition model for home surveillance.

- Dataset Expansion:** To address the limited dataset size, future work should focus on collecting a larger and more diverse dataset that includes a wider variety of vehicle types and conditions. This could involve gathering images from different environments and times of day to better represent real-world scenarios.
- Resource Optimization:** Exploring ways to optimize computational resources will be crucial. This could involve implementing more efficient algorithms or utilizing cloud-based solutions to handle the demands of larger datasets and more complex models.
- Vehicle Detection Model:** As our current dataset only contains images of four types of vehicles, it is essential to develop an additional model that can detect the presence of vehicles in images. This model would help mitigate confusion in the prediction process when vehicles are absent, thereby improving the overall accuracy of the surveillance system.
- Integration of Advanced Techniques:** Future work could also explore the integration of advanced techniques such as ensemble learning or hybrid models that combine the strengths of different architectures. This approach may enhance the robustness and accuracy of the vehicle recognition system in various conditions.
- Adaptation on real environment:** During the project, the whole testing setup is within an emulated environment. However, capturing images with home surveillance cameras may affect image quality, and network latency and potential connectivity issues may hinder the system performance. Insufficient resources on low-ended computers may also affect latency or accuracy of the prediction. Therefore, further testing is needed to understand what the potential needs for our HomeGuard application are.

10 Conclusion

The HomeGuard project successfully demonstrates the feasibility of implementing a lightweight, efficient vehicle surveillance system for residential areas. Through comprehensive testing and evaluation of different neural network architectures, we found that MobileNetV2 provides the optimal balance between accuracy and computational efficiency, achieving 98% validation accuracy,97% on F1-score on testing set, while maintaining reasonable latency even in resource-constrained environments. The simulation results showing average prediction times of few seconds confirm the system's viability for real-time monitoring applications. Our findings highlight that simpler architectures can outperform more complex models when working with limited datasets and computational resources. The hyperparameter tuning results demonstrate that careful optimization of model parameters is crucial for achieving robust performance in practical applications. While the current implementation has certain limitations regarding dataset size and variety, the system provides a solid foundation for future development of accessible home surveillance solutions. This work contributes to the growing field of edge computing and home security by showing how modern deep learning techniques can be effectively deployed on consumer-grade hardware. The proposed

architecture and methodology offer a blueprint for developing similar systems that prioritize both performance and practicality. As smart home technologies continue to evolve, solutions like HomeGuard can play an increasingly important role in enhancing residential security while maintaining user privacy and data control.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to Dr. Qin Lv for her invaluable guidance and support throughout this research project. Her expertise in Data Mining and Machine Learning significantly contributed to shaping the direction and methodology of this work. Special thanks to the open-source community, particularly the contributors of the Vehicle Type Recognition dataset on Kaggle and the developers of various deep learning frameworks that made this research possible.

I am also grateful to the University of Colorado Boulder and Coursera for providing the necessary resources and infrastructure to conduct this research. Finally, I would like to thank my colleagues and family for their continuous encouragement and support throughout this project.

REFERENCES

- [1] 2012 ACM Computing Classification System. Retrieved from <https://dl.acm.org/ccs>.
- [2] *Frigate - NVR With Realtime Object Detection for IP Cameras*. Retrieved from <https://github.com/blakeblackshear/frigate>
- [3] *Vehicle Type Recognition*, Kaggle <https://www.kaggle.com/datasets/kaggleashwin/vehicle-type-recognition/data>
- [4] DeepLabCut. 2025. What neural network should I use? (Trade-offs, speed performance, and considerations). Retrieved from [https://github.com/DeepLabCut/DeepLabCut/wiki/What-neural-network-should-I-use%3F-\(Trade-offs,-speed-performance,-and-considerations\)](https://github.com/DeepLabCut/DeepLabCut/wiki/What-neural-network-should-I-use%3F-(Trade-offs,-speed-performance,-and-considerations)).
- [5] Ahmed, I. 2025. Why your MobileNetV2 model performs better than ResNet50. Medium. Retrieved from <https://medium.com/@imtiazh.ahmed2206/why-your-mobilenetv2-model-performs-better-than-resnet50-2a9998fda4c7>.